

Autonomous Incident Management for Data Pipelines

A Practical 1-Week MVP for a Team of 3

Scope: Demonstrate autonomous incident detection, diagnosis, remediation and validation for a data pipeline using a reusable, configuration-driven architecture with tenant isolation.

1. Problem Statement

Data pipeline failures require engineers to monitor alerts, inspect logs, identify probable root causes, decide corrective actions and verify recovery. The goal is to build a small autonomous incident-management system that can detect a pipeline failure, collect evidence, use an AI agent to diagnose the issue, retrieve similar historical incidents, propose a remediation, execute a safe action, and validate whether the pipeline recovered.

The solution must also avoid tight coupling to a single pipeline technology. A common platform interface and configuration layer will allow the same agent workflow to work with a real or simulated Synapse adapter and a mocked TIBCO adapter. Multiple tenants will use the same application while their configurations and incidents remain logically isolated.

2. Business Use Case

A retail organisation runs daily customer/order data pipelines. When a pipeline fails because of a transient connection or dependency issue, an engineer normally investigates logs and manually retries it. This creates operational delay and consumes engineering time.

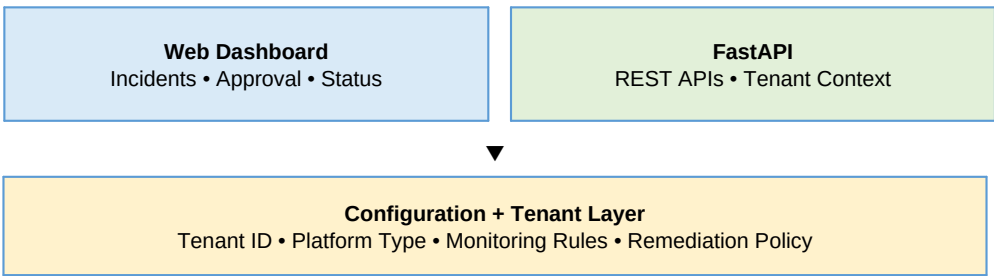
Event	System behaviour
Pipeline fails	Monitoring detects a failed pipeline and creates an incident.
Evidence collection	Retrieve status, logs, recent failures and basic metrics.
AI diagnosis	Identify probable root cause and severity from available evidence.
Historical lookup	Find similar incidents and previous successful actions.
Remediation	Propose a safe action such as retrying the pipeline.
Approval	Apply tenant policy for automatic execution or human approval.
Validation	Check whether the pipeline succeeds after remediation.
Resolution	Close the incident and store the evidence/action trail.

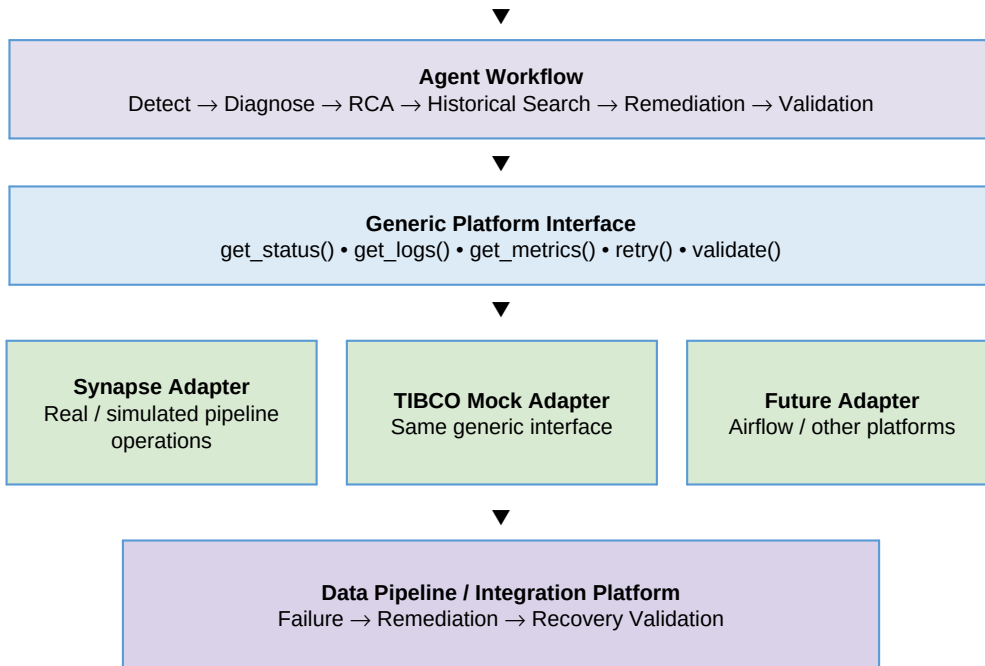
3. Implementation Approach

3.1 MVP Scope

Do not attempt a production-grade AWS/Kubernetes platform in one week. Build a local or single-container demonstrator with clean interfaces. Use one real or simulated pipeline source, one safe remediation action, and one additional mock adapter to prove extensibility.

3.2 Architecture





3.3 Agent Workflow

1	Detection	Detect failed pipeline / receive simulated alert.
2	Diagnosis	Collect status, logs and recent execution information.
3	RCA	LLM produces structured probable root cause with evidence.
4	Historical Search	Retrieve similar incidents from a small knowledge base.
5	Remediation	Select retry as the MVP remediation action.
6	Approval	Apply tenant policy for automatic execution or approval.
7	Execution	Invoke retry through the generic adapter interface.
8	Validation	Check pipeline state and mark resolved/failed.

4. One-Week Implementation Plan

Day	Team focus	Output
Day 1	All 3	Freeze scope; architecture; repo; data model; FastAPI skeleton; configuration format.
Day 2	Person 1	Pipeline monitoring + Synapse-style adapter/simulator.
Day 2	Person 2	Agent workflow: diagnosis, RCA and structured incident output.
Day 2	Person 3	Dashboard + tenant/configuration APIs.
Day 3	Person 1	Generic platform interface + TIBCO mock adapter.
Day 3	Person 2	Historical incident retrieval + remediation planner.
Day 3	Person 3	Incident UI + approval screen + persistence.
Day 4	All 3	Integrate detection → agent → remediation end-to-end.
Day 5	Person 1	Execution + validation + failure handling.
Day 5	Person 2	Prompt tuning, evidence handling and evaluation cases.
Day 5	Person 3	Multi-tenant isolation + audit trail + dashboard metrics.
Day 6	All 3	Testing, bug fixing, demo scenarios and documentation.
Day 7	All 3	Final polish, deployment, architecture presentation and final demo.

5. Deliverables

1. Working FastAPI backend and simple web dashboard.
2. Configuration-driven tenant and platform setup.
3. Generic platform interface / adapter pattern.
4. One Synapse-style adapter connected to a real or simulated pipeline.
5. One TIBCO mock adapter demonstrating extensibility.
6. Agent workflow for detection, diagnosis, RCA, historical search, remediation planning and validation.
7. Small historical incident knowledge base.
8. Safe MVP remediation: pipeline retry.
9. Human approval flow for actions configured as approval-required.
10. Incident persistence and audit log.
11. End-to-end demo with successful retry, failed retry/escalation and cross-platform configuration scenarios.
12. README, architecture diagram, setup guide and short evaluation report.

6. Assumptions

- The 1-week target is a functional MVP/prototype, not a production enterprise platform.
- The team has access to an LLM/API and a Python development environment.
- The pipeline source can be simulated if live Synapse/TIBCO access is unavailable.
- Only one or two remediation actions are implemented; retry is the primary safe action.
- TIBCO is represented by a mock adapter unless a real environment is already available.
- Tenant isolation is logical/application-level for the MVP.
- Monitoring is polling/event based for the demonstration rather than a full enterprise observability deployment.
- Production credentials and secrets are not stored in source code.

7. Future Enhancements

- Add real TIBCO, Airflow, Databricks, ADF and other platform adapters.
- Add remediation actions such as restart, dependency recovery and rollback.
- Introduce risk-based autonomous execution policies.
- Add RAG over historical incidents, runbooks and operational documentation.
- Add event-driven monitoring and enterprise observability integrations.
- Add AWS/Kubernetes deployment, Terraform and CI/CD.
- Add stronger RBAC, secrets management and tenant isolation.
- Learn from remediation outcomes and operator feedback.
- Add MTTR/SLO analytics and richer operational dashboards.
- Add agent tracing and evaluation with LangSmith/LangFuse.

8. Final Demo Success Criteria

The demo should prove one complete incident lifecycle: **failure detected** → **evidence collected** → **AI diagnosis/RCA** → **similar incident found** → **remediation proposed** → **approval/automatic execution** → **adapter call** → **recovery validation** → **incident resolved and audited**. Then switch the tenant/platform configuration from the Synapse-style adapter to the TIBCO mock adapter and show that the same agent workflow remains unchanged. This demonstrates autonomous incident management, abstraction and multi-tenancy within a one-week scope.

9. Recommended MVP Tech Stack

Component	MVP choice
Backend	Python + FastAPI
Agent orchestration	LangGraph
LLM	OpenAI/Azure OpenAI or approved enterprise LLM
Database	SQLite or PostgreSQL
Historical search	SQL/keyword search initially; vector search optional
Frontend	Streamlit or lightweight React
Integration	Adapter/Strategy pattern
Deployment	Docker or local execution
Observability	Application logs; LangSmith/LangFuse if available

Scope decision: The original project asks for monitoring, incident analysis, RCA, historical incident retrieval, remediation, human approval, automated action, validation and auditability. This MVP retains those core capabilities while deliberately reducing infrastructure and integration scope so three people can produce a working demonstration in seven days.